

# Memoria del Proyecto: Biblioteca Municipal

## 1. Descripción General del Proyecto

- **Dominio Elegido:** Gestión de Biblioteca Municipal.
  - **Objetivo:** Desarrollar una aplicación de escritorio robusta para la gestión integral de una biblioteca, automatizando el control de usuarios, el catálogo bibliográfico y el flujo de préstamos.
  - **Funcionalidades Principales:**
    - Gestión de Usuarios (Socios y Bibliotecarios) con seguridad por roles.
    - Administración de Catálogo (CRUD de Libros) con integración de API.
    - Sistema de Préstamos Activos con alertas de retraso.
    - Historial de movimientos y gestión de Reservas (N:M).
    - Interfaz moderna con temas dinámicos y exportación de informes.
-

## 2. Arquitectura y Estructura del Proyecto

La aplicación implementa una arquitectura **MVC (Modelo-Vista-Controlador)** desacoplada mediante el patrón **DAO (Data Access Object)**.

### Explicación de Paquetes:

- **db**: Gestiona la persistencia mediante una conexión Singleton a MySQL.
  - **model**: Contiene las entidades del dominio (POJOs) reflejando la lógica de negocio.
  - **dao**: Capa de abstracción de datos. Define las operaciones CRUD sin exponer la lógica SQL a la vista.
  - **dto**: Objetos de transferencia de datos utilizados para unir información de múltiples tablas (ej: `PrestamoDTO` une Socio y Libro).
  - **view**: Interfaz de usuario Swing, totalmente desacoplada de la lógica de datos.
  - **service**: Lógica de comunicación con servicios externos (API REST).
- 

## 3. Modelo de Base de Datos (Joined Table Inheritance)

Se ha implementado un esquema relacional optimizado que utiliza **herencia de tablas**:

- **Tabla usuarios (Padre)**: Almacena las credenciales básicas (id, username, password, email, rol).
- **Tablas socios y bibliotecarios (Hijas)**: Extienden la tabla `usuarios`. Comparten el mismo ID (Primary Key y Foreign Key al mismo tiempo), permitiendo que un usuario tenga atributos adicionales según su rol sin duplicar datos.
- **Relación N:M (Reservas)**: Resuelta mediante una tabla intermedia que conecta usuarios y libros.
- **Integridad Referencial**: Uso de `ON DELETE CASCADE` para asegurar la limpieza de datos relacionados.

---

## 4. Instrucciones de Instalación y Ejecución

### Configuración del Entorno:

#### Base de Datos (Docker):

`docker-compose up -d`

1. *Nota: El script `database.sql` se importa automáticamente. La base de datos corre en el puerto **3307**.*
2. **Dependencias:** Añadir al Classpath los archivos JAR de la carpeta `lib` (MySQL Connector y FlatLaf).
3. **Ejecución:** Compilar y ejecutar la clase `Main.java`.

---

## 5. Repositorio de GitHub

- **URL:** [PEGA\_AQUÍ\_TU\_ENLACE]
- **Commits:** El historial refleja un desarrollo incremental con commits descriptivos (Frontend, Backend, Integración API, etc.).

---

## 6. Informe WakaTime (ProyectoFinal-Programacion)

- **Tiempo registrado:** Mínimo de 10 horas acreditadas.
  - **Evidencia:** [PEGA\_AQUÍ\_ENLACE\_O\_INDICACIÓN\_DE\_CAPTURA]
-

## 7. Extensiones Implementadas (Opcionales)

- **API REST (Open Library):** Integración con la API pública de Open Library para recuperar datos de libros por ISBN de forma automática, mejorando la experiencia de usuario en el alta de ejemplares.
- **Modo Oscuro/Claro Dinámico:** Cambio de Look & Feel en tiempo de ejecución mediante la librería **FlatLaf**, proporcionando una estética premium y moderna.
- **Exportación de Datos:** Generación de archivos **CSV** desde cualquier tabla del sistema, permitiendo la portabilidad de la información a herramientas como Excel.